# Chapter 5 | Exercise solutions with Python

*Multiple Heads, Position, and the Complete Architecture*

This notebook solves Exercises 1 and 4–12, including both optional challenges. It traces the head axes, works through the sublayers and vocabulary readout, and checks how the complete model handles padding and future target tokens. Exercises 2 and 3 are in the written solutions.

To rerun the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages from `requirements-ch05.txt`. Run the cells in order. The checks use float64 on the CPU. The saved outputs are included.

```
from pathlib import Path
import sys

import torch
from torch import nn
from torch.nn import functional as F

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch05_transformer" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")

torch.set_num_threads(1)
```

## 1. Follow the head axes

Start with one example and two token rows. The first token carries $(1, 2, 3, 4)$ and the second carries $(5, 6, 7, 8)$. We divide model width 4 among two heads, so the width per head is $4/2 = 2$. Within each token, head 1 gets its first two coordinates and head 2 gets its last two.

| Head, counted from 1 | Token 1 | Token 2 |
|---|---|---|
| 1 | $(1, 2)$ | $(5, 6)$ |
| 2 | $(3, 4)$ | $(7, 8)$ |

The input axes are `[batch, token, model feature]`. Reshape the last axis of length 4 into two axes of length 2. The result has axes `[batch, token, head, head feature]`. At this stage, indexing the second axis still selects a token and gives both of that token's heads. Transpose the token and head axes to obtain `[batch, head, token, head feature]`. Now indexing the second axis gives the two tokens for one head, as shown in the table.

Using zero-based indices $b$ for example, $h$ for head, $t$ for token, and $j$ for head feature, the element at $[b, h, t, j]$ in the split tensor is the input at $[b, t, 2h + j]$. In particular, head 1's second token takes the first two coordinates of $(5, 6, 7, 8)$.

```
from examples.ch05_transformer.model import MultiHeadAttention

x1 = torch.tensor(
    [[[1, 2, 3, 4], [5, 6, 7, 8]]], dtype=torch.float64
)
attention1 = MultiHeadAttention(4, 2).double().eval()
split1 = attention1.split_heads(x1)
expected1 = x1.new_tensor(
    [[[1, 2], [5, 6]], [[3, 4], [7, 8]]]
)
assert split1.shape == (1, 2, 2, 2)
torch.testing.assert_close(split1, expected1, rtol=0, atol=0)
for head1 in range(2):
    print(f"Head {head1 + 1}, one row per token:")
    print(split1[0, head1].tolist())
```

OUTPUT

```
Head 1, one row per token:
[[1.0, 2.0], [5.0, 6.0]]
Head 2, one row per token:
[[3.0, 4.0], [7.0, 8.0]]
```

To merge, transpose the token and head axes back. Each token now holds its two head pieces consecutively. Reshape the head and head-feature axes into one width-four feature axis. This should recover the input entries in their original order.

The two layouts have the same numerical shape because the token and head axes both have length two. To see what a shape check would miss, pass the reshape-only tensor to `merge_heads` as though its second axis were the head axis. Its first two rows under that interpretation would be `[1, 2]` and `[3, 4]`, two pieces of the first token. That incorrect grouping sends different entries into the merge.

```
recovered1 = attention1.merge_heads(split1)
assert recovered1.shape == x1.shape == (1, 2, 4)
torch.testing.assert_close(recovered1, x1, rtol=0, atol=0)

reshape_only1 = x1.reshape(1, 2, 2, 2)
wrong_merge1 = attention1.merge_heads(reshape_only1)
assert reshape_only1.shape == split1.shape
assert not torch.equal(reshape_only1, split1)
wrong_expected1 = x1.new_tensor(
    [[[1, 2, 5, 6], [3, 4, 7, 8]]]
)
torch.testing.assert_close(wrong_merge1, wrong_expected1)
print("Correct merge:", recovered1.tolist())
print("Reshape-only grouping:", reshape_only1[0].tolist())
print("Merge of reshape-only grouping:", wrong_merge1.tolist())
```

OUTPUT

```
Correct merge: [[[1.0, 2.0, 3.0, 4.0], [5.0, 6.0, 7.0, 8.0]]]
Reshape-only grouping: [[[1.0, 2.0], [3.0, 4.0]], [[5.0, 6.0], [7.0, 8.0]]]
Merge of reshape-only grouping: [[[1.0, 2.0, 5.0, 6.0], [3.0, 4.0, 7.0, 8.0]]]
```

## 4. Inspect a post-norm wrapper

The wrapper applies dropout to the sublayer's update, adds that result to the incoming vector, and normalizes the sum. Here the update is zero, so the dropout output is zero. For incoming vector $\mathbf{x} = (1, 2, 3, 4)$, the residual sum is $\mathbf{r} = \mathbf{x} + (0, 0, 0, 0) = (1, 2, 3, 4)$.

LayerNorm works across the four features of this one vector. Its mean is $\mu = (1 + 2 + 3 + 4)/4 = 2.5$. The centered features are $(-1.5, -0.5, 0.5, 1.5)$. Their squared values are $(2.25, 0.25, 0.25, 2.25)$, so the population variance is $\sigma^2 = (2.25 + 0.25 + 0.25 + 2.25)/4 = 1.25$. The divisor counts all four features.

With scale one and offset zero at each feature, the answer is

$$\frac{(-1.5, -0.5, 0.5, 1.5)}{\sqrt{1.25 + 10^{-5}}} \approx (-1.341635, -0.447212, 0.447212, 1.341635).$$

The residual addition carried the input into the sum. The normalization that follows centers and rescales that sum, so the whole wrapper changes the vector even with zero update.

For incoming vector $(3, 3, 3, 3)$, the residual sum is $(3, 3, 3, 3)$, its mean is $3$, and every centered feature is zero. The variance is zero. The denominator is $\sqrt{10^{-5}} > 0$, so division produces a zero vector and the affine transformation with scale one and offset zero leaves it at $(0, 0, 0, 0)$. The added constant keeps the computation defined in this case.

```
residual4 = torch.tensor(
    [[1, 2, 3, 4], [3, 3, 3, 3]], dtype=torch.float64
)
update4 = torch.zeros_like(residual4)
summed4 = residual4 + update4
eps4 = 1e-5
layer_norm4 = nn.LayerNorm(4, eps=eps4).double().eval()

centered4 = summed4 - summed4.mean(dim=-1, keepdim=True)
variance4 = centered4.square().mean(dim=-1, keepdim=True)
manual4 = centered4 / (variance4 + eps4).sqrt()
with torch.no_grad():
    normalized4 = layer_norm4(summed4)

torch.testing.assert_close(
    layer_norm4.weight, torch.ones_like(layer_norm4.weight)
)
torch.testing.assert_close(
    layer_norm4.bias, torch.zeros_like(layer_norm4.bias)
)
torch.testing.assert_close(
    variance4, residual4.new_tensor([[1.25], [0.0]])
)
torch.testing.assert_close(normalized4, manual4, rtol=0, atol=1e-14)
torch.testing.assert_close(normalized4[1], update4[1], rtol=0, atol=0)
for row4, output4 in zip(summed4.tolist(), normalized4.tolist()):
    print(f"Residual sum {row4}")
    print("Post-norm output", [round(value, 6) for value in output4])
```

OUTPUT

```
Residual sum [1.0, 2.0, 3.0, 4.0]
Post-norm output [-1.341635, -0.447212, 0.447212, 1.341635]
Residual sum [3.0, 3.0, 3.0, 3.0]
Post-norm output [0.0, 0.0, 0.0, 0.0]
```

## 5. Track dropout values

During training with drop probability $p = 1/2$, a coordinate $x_j$ is dropped with probability $1/2$. If it is retained, the layer divides it by the keep probability $1 - p = 1/2$. Write $M_j = 0$ or $1$ for the random choice. The output is $M_j x_j/(1 - p) = 2M_j x_j$.

| Input coordinate | Dropped, probability $1/2$ | Retained, probability $1/2$ | Expected output |
|---|---|---|---|
| 1 | 0 | 2 | $\frac{1}{2}(0) + \frac{1}{2}(2) = 1$ |
| 2 | 0 | 4 | $\frac{1}{2}(0) + \frac{1}{2}(4) = 2$ |
| 3 | 0 | 6 | $\frac{1}{2}(0) + \frac{1}{2}(6) = 3$ |
| 4 | 0 | 8 | $\frac{1}{2}(0) + \frac{1}{2}(8) = 4$ |

The expected vector is $(1, 2, 3, 4)$. That average describes many possible masks. One training call yields one sample, with each coordinate either zero or twice its input. In evaluation mode, dropout returns $(1, 2, 3, 4)$ on every call.

The following check keeps the module in training mode for two calls. It resets the CPU random seed to produce the same mask in both. The first call records gradients for the floating-point input, and the second runs inside `torch.no_grad()`. The input requests gradients so the output flags distinguish those two contexts. The final call switches the module to evaluation mode and runs with gradient recording enabled.

```
x5 = torch.tensor(
    [1, 2, 3, 4], dtype=torch.float64, requires_grad=True
)
dropout5 = nn.Dropout(p=0.5).train()
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(7)
    with torch.enable_grad():
        trained5 = dropout5(x5)
    torch.manual_seed(7)
    with torch.no_grad():
        unrecorded5 = dropout5(x5)
assert dropout5.training

dropout5.eval()
with torch.enable_grad():
    evaluated5 = dropout5(x5)

assert torch.all((trained5 == 0) | (trained5 == 2 * x5))
torch.testing.assert_close(trained5, unrecorded5, rtol=0, atol=0)
torch.testing.assert_close(evaluated5, x5, rtol=0, atol=0)
assert trained5.requires_grad and evaluated5.requires_grad
assert not unrecorded5.requires_grad

print("Training:", trained5.tolist(), trained5.requires_grad)
print("Training with no_grad:", unrecorded5.tolist(),
      unrecorded5.requires_grad)
print("Evaluation:", evaluated5.tolist(), evaluated5.requires_grad)
print("The final flag on each line is requires_grad.")
```

```
OUTPUT
Training: [2.0, 4.0, 0.0, 0.0] True
Training with no_grad: [2.0, 4.0, 0.0, 0.0] False
Evaluation: [1.0, 2.0, 3.0, 4.0] True
The final flag on each line is requires_grad.
```

In the saved training sample the first two coordinates double and the last two become zero. Training mode controls the dropout rule. Gradient recording controls whether PyTorch tracks differentiable operations. Both training calls sample and rescale values, even though only one records gradients. Evaluation switches dropout to its identity rule, and the evaluated result retains gradient tracking through the input. These two controls answer different questions.

## 6. Follow both heads.

Start with the second input row, $(0, 1, 0, 2)$. The query and key projections are identities. Head 1 receives the first two projected coordinates and head 2 receives the last two. The value projection swaps those two coordinate pairs, so head 2 reads the first pair from each input row as its value.

For head 2 the second query is therefore $(0, 2)$. Write its keys and values in the same token order as the input. The key and value on a row refer to the same position.

| Position | Head 2 key | Head 2 value | Dot product with $(0, 2)$ |
|---|---|---|---|
| 1 | $(1, 0)$ | $(1, 0)$ | $0(1) + 2(0) = 0$ |
| 2 | $(0, 2)$ | $(0, 1)$ | $0(0) + 2(2) = 4$ |
| 3 | $(1, 1)$ | $(1, 1)$ | $0(1) + 2(1) = 2$ |

The key width of one head is two, so the scaled scores are $(0, 2\sqrt{2}, \sqrt{2})$. Define $r = \exp(\sqrt{2})$ and $D_2 = 1 + r^2 + r$. The second row of head 2's attention map is

$$\left(\frac{1}{D_2}, \frac{r^2}{D_2}, \frac{r}{D_2}\right) \approx (0.045388,\ 0.767918,\ 0.186694).$$

Each weight scales the value at its key position. The three scaled values are $(1/D_2, 0)$, $(0, r^2/D_2)$, and $(r/D_2, r/D_2)$. Summing each feature separately gives

$$\mathbf{o}_2^{(2)} = \left(\frac{1 + r}{D_2}, \frac{r^2 + r}{D_2}\right) \approx (0.232082,\ 0.954612).$$

The most heavily weighted value is $(0, 1)$ at position 2. This contributes much of the second coordinate and contributes zero to the first.

Head 1's second query is $(0, 1)$. Against its keys $(1, 0), (0, 1), (1, 1)$ the raw score row is $(0, 1, 1)$. Let $u = \exp(1/\sqrt{2})$ and $D_1 = 1 + 2u$. The weight row is $(1, u, u)/D_1$. Its values are $(1, 0), (0, 2), (1, 1)$, so the read is $((1 + u)/D_1, 3u/D_1) \approx (0.598888, 1.203336)$.

Both reads belong to query position 2. Concatenate their feature coordinates in head order to obtain $(a, b, c, d) \approx (0.598888, 1.203336, 0.232082, 0.954612)$. The output projection maps the row to $(a + c/2,\ b + d/2,\ a/2 + c,\ b/2 + d)$. Using full-precision values gives

$$(0.714929,\ 1.680642,\ 0.531526,\ 1.556280) \quad \text{to six decimal places.}$$

The output projection combines features at this one position. The weighted sums inside the heads combined values across positions.

The returned attention tensor has shape `[1, 2, 3, 3]`. Its axes are batch, head, query position, and key position. In code, `weights[0, 1, 1]` therefore selects all key weights for head 2 at query position 2. The returned concatenated and projected outputs both have shape `[1, 3, 4]`, with axes batch, query position, and feature.

```
import math

from examples.ch05_transformer.model import (
    MultiHeadAttention,
    PositionwiseFeedForward,
)
from examples.ch05_transformer.run_examples import (
    toy_feed_forward,
    toy_multihead,
    vocabulary_readout_example,
)

component_dtype = torch.float64

def format_component_row(row):
    return "[" + ", ".join(f"{item:.6f}" for item in row) + "]"

raw_head_2 = torch.tensor([0., 4., 2.], dtype=component_dtype)
head_2_values = torch.tensor(
    [[1., 0.], [0., 1.], [1., 1.]], dtype=component_dtype
)
exponents_head_2 = (raw_head_2 / math.sqrt(2)).exp()
weights_head_2 = exponents_head_2 / exponents_head_2.sum()
head_2_read = weights_head_2 @ head_2_values

u = math.exp(1 / math.sqrt(2))
denominator_head_1 = 1 + 2 * u
head_1_read = torch.tensor(
    [1 + u, 3 * u], dtype=component_dtype
) / denominator_head_1

hand_concat = torch.cat((head_1_read, head_2_read))
a, b, c, d = hand_concat.unbind()
hand_output = torch.stack((a + c / 2, b + d / 2, a / 2 + c,
                           b / 2 + d))
```

```
toy_output, toy_weights, toy_concat = toy_multihead()
assert toy_weights.shape == (1, 2, 3, 3)
assert toy_output.shape == toy_concat.shape == (1, 3, 4)
torch.testing.assert_close(weights_head_2, toy_weights[0, 1, 1])
torch.testing.assert_close(hand_concat, toy_concat[0, 1])
torch.testing.assert_close(hand_output, toy_output[0, 1])

torch.testing.assert_close(toy_weights[0, 0, 0], toy_weights[0, 1, 0])
assert not torch.allclose(toy_weights[0, 0, 1:], toy_weights[0, 1, 1:])
print("Raw head 2 scores:", raw_head_2.tolist())
print("Head 2 weights:", format_component_row(weights_head_2))
print("Head 1 read:", format_component_row(head_1_read))
print("Head 2 read:", format_component_row(head_2_read))
print("Concatenated:", format_component_row(hand_concat))
print("Projected:", format_component_row(hand_output))
```

```
OUTPUT
Raw head 2 scores: [0.0, 4.0, 2.0]
Head 2 weights: [0.045388, 0.767918, 0.186694]
Head 1 read: [0.598888, 1.203336]
Head 2 read: [0.232082, 0.954612]
Concatenated: [0.598888, 1.203336, 0.232082, 0.954612]
Projected: [0.714929, 1.680642, 0.531526, 1.556280]
```

The first raw score row happens to be $(1, 0, 1)$ in both heads. In head 1 and head 2 the first query is $(1, 0)$ and the first key coordinates across positions are $1, 0, 1$. This makes the first weight rows identical. The second query gives $(0, 1, 1)$ in head 1 and $(0, 4, 2)$ in head 2. Normalizing those rows gives different distributions. Each row describes one query's comparisons, so agreement at the first position says nothing about the later queries.

## 7. Separate concatenation from mixing.

Copy the setup from `toy_multihead` and use the identity for $W^O$. The queries, keys, and values still come from the same input and the same three projection matrices. This gives the original attention weights and the original head reads. Concatenation places the head reads side by side at each position, so that tensor is also unchanged. Multiplying the concatenation by the identity leaves it as is. Of the three values returned by the helper, only the projected output changes.

In PyTorch, a bias-free `nn.Linear` stores its weight with axes output feature, input feature. The matrix used for row-vector multiplication in the book has axes input feature, output feature. Copy the transpose of the book's value matrix into `v_proj.weight`. The two identity matrices for queries and keys and the new identity for the readout all equal their own transposes.

```
x_identity = torch.tensor(
    [[[1., 0., 1., 0.], [0., 1., 0., 2.], [1., 1., 1., 1.]]],
    dtype=component_dtype,
)
w_value = torch.tensor(
    [[0., 0., 1., 0.], [0., 0., 0., 1.],
     [1., 0., 0., 0.], [0., 1., 0., 0.]],
    dtype=component_dtype,
)
identity_4 = torch.eye(4, dtype=component_dtype)
identity_attention = MultiHeadAttention(4, 2).double().eval()
with torch.no_grad():
    identity_attention.q_proj.weight.copy_(identity_4)
    identity_attention.k_proj.weight.copy_(identity_4)
    identity_attention.v_proj.weight.copy_(w_value.T)
    identity_attention.out_proj.weight.copy_(identity_4)
    new_output, new_weights = identity_attention(
        x_identity, x_identity, x_identity
    )
    split_values = identity_attention.split_heads(
        identity_attention.v_proj(x_identity)
    )
    new_concat = identity_attention.merge_heads(
        new_weights @ split_values
    )

torch.testing.assert_close(new_weights, toy_weights)
torch.testing.assert_close(new_concat, toy_concat)
torch.testing.assert_close(new_output, new_concat)
assert not torch.allclose(new_output, toy_output)

original_row = format_component_row(toy_output[0, 1])
identity_row = format_component_row(new_output[0, 1])
print("Position 2, original output:", original_row)
print("Position 2, identity output:", identity_row)
print("Weights and concatenation agree at every position.")
```

```
OUTPUT
Position 2, original output: [0.714929, 1.680642, 0.531526, 1.556280]
Position 2, identity output: [0.598888, 1.203336, 0.232082, 0.954612]
Weights and concatenation agree at every position.
```

Fix the model width at eight. With two heads, each head gets four output features from each input projection. The query projection can be viewed as two $8 \times 4$ matrices side by side, giving $2(8)(4) = 64$ parameters. Keys and values each add 64. With four heads, the query projection is four $8 \times 2$ matrices side by side and still has $4(8)(2) = 64$ entries. The same holds for keys and values.

Concatenation yields eight features in both cases. The output projection mixes them into an eight-feature output and has $8(8) = 64$ entries. All four projections are bias-free.

| Heads | Width per head | Query | Key | Value | Output | Total |
|---|---|---|---|---|---|---|
| 2 | 4 | 64 | 64 | 64 | 64 | 256 |
| 4 | 2 | 64 | 64 | 64 | 64 | 256 |

Changing the number of heads changes how the projected features are partitioned and how many attention maps are computed. It leaves the full projection sizes fixed when the total model width is fixed.

```
for n_heads in (2, 4):
    attention = MultiHeadAttention(8, n_heads).double().eval()
    sizes = [p.numel() for p in attention.parameters()]
    assert sizes == [64, 64, 64, 64]
    print(f"{n_heads} heads, width {attention.head_width} per head")
    print(f"  Q, K, V, O counts: {sizes}. Total: {sum(sizes)}.")
```

```
OUTPUT
2 heads, width 4 per head
  Q, K, V, O counts: [64, 64, 64, 64]. Total: 256.
4 heads, width 2 per head
  Q, K, V, O counts: [64, 64, 64, 64]. Total: 256.
```

## 8. Test the positionwise FFN.

In the toy feed-forward network, a row first expands from width four to eight. With zero bias, its first four entries are the original row and its last four entries are their negatives. For $x = (1, -2, 0, 3)$ this gives $(1, -2, 0, 3, -1, 2, 0, -3)$. ReLU replaces negative entries with zero, giving $(1, 0, 0, 3, 0, 2, 0, 0)$.

The second linear map adds the first group of four to the second group coordinate by coordinate. It gives $(1, 2, 0, 3)$. For any coordinate $x_i$ of an input row, the resulting function is $\max(0, x_i) + \max(0, -x_i) = |x_i|$ for these particular weights.

Construct a fresh module. The expansion stores $[I_4; -I_4]$ as an $8 \times 4$ weight array and the contraction stores $[I_4\ I_4]$ as a $4 \times 8$ weight array, since the output feature axis comes first in the stored weights. The canonical helper returns the numbers for a single input and can check our work.

```
positionwise = PositionwiseFeedForward(4, 8).double().eval()
input_row = torch.tensor([1., -2., 0., 3.], dtype=component_dtype)
with torch.no_grad():
    positionwise.expand.weight.copy_(
        torch.cat((identity_4, -identity_4), dim=0)
    )
    positionwise.expand.bias.zero_()
    positionwise.contract.weight.copy_(
        torch.cat((identity_4, identity_4), dim=1)
    )
    positionwise.contract.bias.zero_()
    expanded = positionwise.expand(input_row)
    hidden = expanded.relu()
    ffn_output = positionwise(input_row)

canonical_hidden, canonical_ffn_output = toy_feed_forward()
torch.testing.assert_close(hidden, canonical_hidden)
torch.testing.assert_close(ffn_output, canonical_ffn_output)
torch.testing.assert_close(ffn_output, input_row.abs())
print("Expanded:", expanded.tolist())
print("After ReLU:", hidden.tolist())
print("After contraction:", ffn_output.tolist())
```

OUTPUT

```
Expanded: [1.0, -2.0, 0.0, 3.0, -1.0, 2.0, 0.0, -3.0]
After ReLU: [1.0, 0.0, 0.0, 3.0, 0.0, 2.0, 0.0, 0.0]
After contraction: [1.0, 2.0, 0.0, 3.0]
```

Put three different vectors in a tensor of shape `[1, 3, 4]`. The axes index the batch item, token position, and features. Both linear maps operate on the final feature axis, and ReLU operates on each entry. There is no reduction over the token axis. Changing the token order therefore changes the output order in the same way. Replacing just token 2 leaves the outputs for tokens 1 and 3 as they were.

```
ffn_rows = torch.tensor(
    [[[1., -2., 0., 3.], [0., 3., -1., 2.], [-4., 0., 2., -1.]]],
    dtype=component_dtype,
)
reordered = [2, 0, 1]
changed_rows = ffn_rows.clone()
changed_rows[0, 1, 1] = -5.
with torch.no_grad():
    original_ffn = positionwise(ffn_rows)
    permuted_ffn = positionwise(ffn_rows[:, reordered, :])
    changed_ffn = positionwise(changed_rows)

torch.testing.assert_close(original_ffn, ffn_rows.abs())
torch.testing.assert_close(permuted_ffn, original_ffn[:, reordered, :])
torch.testing.assert_close(changed_ffn[:, [0, 2], :],
                           original_ffn[:, [0, 2], :])
assert not torch.equal(changed_ffn[:, 1, :], original_ffn[:, 1, :])
print(f"{'Original':22}{'Permuted [3,1,2]':22}Changed row 2")
for old, moved, new in zip(original_ffn[0], permuted_ffn[0],
                           changed_ffn[0]):
    print(f"{str(old.tolist()):22}{str(moved.tolist()):22}"
          f"{new.tolist()}")
```

OUTPUT

```
Original              Permuted [3,1,2]      Changed row 2
[1.0, 2.0, 0.0, 3.0]  [4.0, 0.0, 2.0, 1.0]  [1.0, 2.0, 0.0, 3.0]
[0.0, 3.0, 1.0, 2.0]  [1.0, 2.0, 0.0, 3.0]  [0.0, 5.0, 1.0, 2.0]
[4.0, 0.0, 2.0, 1.0]  [0.0, 3.0, 1.0, 2.0]  [4.0, 0.0, 2.0, 1.0]
```

## 9. Read a vocabulary distribution.

Use the separate toy decoder row $y = (1, -1, 0, 2)$ and zero bias. Each logit is the dot product of $y$ with one column of the vocabulary weight matrix. The vocabulary order is *el*, *coche*, EOS.

| Token | Column | Dot product with $y$ | Logit |
|---|---|---|---|
| *el* | $(1, 0, 0, 1/2)$ | $1(1) + (-1)(0) + 0(0) + 2(1/2)$ | 2 |
| *coche* | $(0, -1, 0, 0)$ | $1(0) + (-1)(-1) + 0(0) + 2(0)$ | 1 |
| EOS | $(-1, 0, 1, 1/2)$ | $1(-1) + (-1)(0) + 0(1) + 2(1/2)$ | 0 |

Exponentiating produces $(e^2, e, 1)$. Their sum $D = e^2 + e + 1$ normalizes the three vocabulary entries. The probability row is $(e^2/D, e/D, 1/D)$, approximately $(0.665241, 0.244728, 0.090031)$.

The loss for the reference word *coche* uses its probability $e/D$, before rounding. The negative natural logarithm is

$$-\log(e/D) = \log D - 1 = \log(e + 1 + e^{-1}) \approx 1.407605964444380.$$

The logits for this single prediction have shape `[1, 3]`. The first axis counts the one prediction row and the final axis indexes the vocabulary. The target is a length-one tensor containing index 1, the zero-based index of *coche* among these three classes. There is no source-position or head axis in this tensor. `F.cross_entropy` accepts logits because it performs the log normalization internally.

```
logits, probabilities = vocabulary_readout_example()
assert logits.shape == probabilities.shape == (1, 3)
torch.testing.assert_close(
    logits, torch.tensor([[2., 1., 0.]], dtype=component_dtype)
)

normalizer = math.exp(2) + math.exp(1) + 1
hand_probabilities = torch.tensor(
    [[math.exp(2), math.exp(1), 1.]], dtype=component_dtype
) / normalizer
hand_nll = math.log(normalizer) - 1
torch_nll = F.cross_entropy(logits, torch.tensor([1]))
torch.testing.assert_close(probabilities, hand_probabilities)
torch.testing.assert_close(torch_nll, -probabilities[0, 1].log())
torch.testing.assert_close(
    torch_nll, torch.tensor(hand_nll, dtype=component_dtype)
)
print("Logits:", logits[0].tolist())
print("Probabilities:", format_component_row(probabilities[0]))
print(f"Hand loss:          {hand_nll:.15f}")
print(f"Cross-entropy loss: {torch_nll.item():.15f}")
```

```
OUTPUT
Logits: [2.0, 1.0, 0.0]
Probabilities: [0.665241, 0.244728, 0.090031]
Hand loss:          1.407605964444380
Cross-entropy loss: 1.407605964444380
```

## 10. Keep query validity separate from key validity

In the shorter source sentence, the last two entries are PAD. Those entries occupy space in the batch so that both sentences have width five. Change the IDs in the padding slots and keep their validity flags fixed. If a real target position is insulated from those slots, all twelve of its vocabulary logits should agree before and after the change.

The code recreates the one-layer demonstration with seed 7, model width four, two heads, and FFN width eight. It uses the same twelve-entry source and target vocabularies. Evaluation mode disables dropout. The two forward passes in this exercise only inspect values, so they also run with gradient recording off.

```
from examples.ch05_transformer.model import Transformer, build_masks
from examples.ch05_transformer.run_examples import (
    check_prefix_and_padding,
    example_batch,
)

def make_exercise_transformer(cls=Transformer, layers=1):
    with torch.random.fork_rng(devices=[]):
        torch.manual_seed(7)
        model = cls(
            12, 12, d_model=4, n_heads=2, d_ff=8,
            n_encoder_layers=layers, n_decoder_layers=layers,
        ).double()
    return model.eval()

model10 = make_exercise_transformer()
source10, target10, labels10 = example_batch()
source_valid10 = source10.ne(model10.pad_id)
target_valid10 = target10.ne(model10.pad_id)
label_valid10 = labels10.ne(model10.pad_id)
assert torch.equal(target_valid10, label_valid10)

changed_source10 = source10.clone()
changed_source10[~source_valid10] = 11
print("Original source:", source10.tolist())
print("Changed source: ", changed_source10.tolist())
print("Saved validity: ", source_valid10.int().tolist())
```

```
OUTPUT
Original source: [[3, 4, 5, 2, 0], [6, 5, 2, 0, 0]]
Changed source:  [[3, 4, 5, 2, 11], [6, 5, 2, 11, 11]]
Saved validity:  [[1, 1, 1, 1, 0], [1, 1, 1, 0, 0]]
```

The saved Boolean source mask marks **key slots**. The encoder uses it at every source query, and the decoder uses it for every cross-attention query. In either place the mask broadcasts from shape $[B, 1, 1, S]$ over heads and queries. Disallowed key scores are set to $-\infty$ before softmax, so their weights are zero.

A padded source query can still produce a hidden vector. In an encoder layer, its vector remains in a masked key slot in the next layer. The decoder also masks that slot when it reads the final encoder memory. Residual connections, LayerNorm, and FFNs operate within each token row, so these sublayers cannot move the padded vector into a valid source row. Recomputing validity from the edited IDs would mark ID 11 as real and change the experiment. Pass the original flags to both forward passes.

```
with torch.no_grad():
    before10, maps10 = model10(
        source10, target10, source_valid=source_valid10,
        return_weights=True,
    )
    after10 = model10(
        changed_source10, target10, source_valid=source_valid10,
    )

before_valid10 = before10[target_valid10]
after_valid10 = after10[target_valid10]
torch.testing.assert_close(
    after_valid10, before_valid10, atol=1e-12, rtol=0,
)
change10 = (after_valid10 - before_valid10).abs().max().item()
print("Valid target logits shape:", tuple(before_valid10.shape))
print(f"Maximum change in those logits: {change10:.3e}")

pad_logits10 = before10[~target_valid10]
assert torch.count_nonzero(pad_logits10) > 0
print("Padded query's 12 logits, printed six per row:")
print(pad_logits10.reshape(2, 6))
```

```
OUTPUT
Valid target logits shape: (7, 12)
Maximum change in those logits: 0.000e+00
Padded query's 12 logits, printed six per row:
tensor([[ 0.3070, -0.0911, -0.4859,  0.3502, -1.2396, -0.3378],
        [-1.2766,  1.1746, -0.5014, -0.1086,  0.8849,  0.1755]],
       dtype=torch.float64)
```

The batch has seven valid target queries. The last query of the second example occupies a padded slot. Its decoder self-attention can still read the three valid target keys that precede it. Its cross-attention can also read the three real source keys. The final linear projection produces a logit for each vocabulary entry at this slot as it does at a real query.

The label at that padded query is PAD, ID 0. The training loss uses `ignore_index=0`, so the query contributes neither a loss term nor one of the positions in the mean. For the seven remaining labels, cross-entropy still normalizes over the whole target vocabulary. A key mask controls what a query can read. The label determines whether the loss scores its output. In this particular batch the valid input slots and the scored label slots coincide.

Verify that filtering to seven rows before computing the loss gives the same result as flattening all eight rows and using `ignore_index`. Then assign a different vector of logits to the ignored row and compare the loss again.

```
pad_self10 = maps10["decoder"][0][1, 0, -1]
pad_cross10 = maps10["cross"][0][1, 0, -1]
assert pad_self10[-1].item() == 0.0
assert torch.count_nonzero(pad_cross10[3:]).item() == 0
self_weights10 = [round(weight, 4) for weight in pad_self10.tolist()]
cross_weights10 = [round(weight, 4) for weight in pad_cross10.tolist()]
print("PAD query, head 1, self: ", self_weights10)
print("PAD query, head 1, cross:", cross_weights10)

loss10 = F.cross_entropy(
    before10.reshape(-1, 12), labels10.flatten(), ignore_index=0,
)
filtered10 = F.cross_entropy(
    before10[label_valid10], labels10[label_valid10],
)
edited_logits10 = before10.clone()
edited_logits10[~label_valid10] = torch.linspace(
    -50.0, 50.0, 12, dtype=before10.dtype,
)
edited_loss10 = F.cross_entropy(
    edited_logits10.reshape(-1, 12),
    labels10.flatten(),
    ignore_index=0,
)
torch.testing.assert_close(loss10, filtered10, atol=1e-12, rtol=0)
torch.testing.assert_close(edited_loss10, loss10, atol=1e-12, rtol=0)
print(f"Loss over {label_valid10.sum().item()} valid labels:")
print(f"  With ignore_index     {loss10.item():.12f}")
print(f"  With filtered rows    {filtered10.item():.12f}")
print(f"  With edited PAD row   {edited_loss10.item():.12f}")
```

OUTPUT

```
PAD query, head 1, self:  [0.3414, 0.3714, 0.2873, 0.0]
PAD query, head 1, cross: [0.3012, 0.2729, 0.4259, 0.0, 0.0]
Loss over 7 valid labels:
  With ignore_index     2.973556394290
  With filtered rows    2.973556394290
  With edited PAD row   2.973556394290
```

## 11. Break causality deliberately (optional challenge)

Consider the first target example, whose decoder inputs are `[1, 8, 9, 10]`. Change just its third input from 9 to 6. With the causal mask, query position 1 may read only key position 1. Query position 2 may read key positions 1 and 2. Neither can read the changed key at position 3.

Create a temporary copy of the model and remove the lower-triangular factor from the target mask assignment. The edited version permits any valid target key at every query. It still masks PAD keys. The source-mask expression stays the same, since source validity and target causality are separate. Import the temporary file under a distinct module name and create its `Transformer` class with the same weights as the canonical model. Now the mask is the only difference between the two models.

```
from importlib.util import module_from_spec, spec_from_file_location
from tempfile import TemporaryDirectory

model_file11 = (
    repo_root / "examples" / "ch05_transformer" / "model.py"
)
original11 = model_file11.read_text(encoding="utf-8")
causal_assignment11 = (
    "target_allowed = causal[None, None] "
    "& target_valid[:, None, None, :]"
)
valid_only_assignment11 = (
    "target_allowed = target_valid[:, None, None, :]"
)
assert original11.count(causal_assignment11) == 1

with TemporaryDirectory(prefix="ch05-mask-") as temporary11:
    experimental_file11 = Path(temporary11) / "model.py"
    experimental_file11.write_text(
        original11.replace(
            causal_assignment11, valid_only_assignment11, 1,
        ),
        encoding="utf-8",
    )
    spec11 = spec_from_file_location(
        "ch05_keys_only_experiment", experimental_file11,
    )
    experimental11 = module_from_spec(spec11)
    spec11.loader.exec_module(experimental11)

causal11 = make_exercise_transformer()
keys_only11 = make_exercise_transformer(experimental11.Transformer)
keys_only11.load_state_dict(causal11.state_dict())
```

Print the allowed keys for the first query of the shorter target. The keys-only row broadcasts to every query. It permits the two future real keys while keeping the final padding key excluded. In the original model, the first query reads only its own key.

Next, use both target sequences with each model. Measure how much the first two logit vectors of the edited example change. The maximum absolute difference summarizes each twelve-entry vector. The assertions compare the complete vectors for the causal model and require a measurable change at both positions in the keys-only model.

```
source11, target11, _ = example_batch()
source_valid11 = source11.ne(causal11.pad_id)
target_valid11 = target11.ne(causal11.pad_id)
source_mask11, causal_mask11 = build_masks(
    source_valid11, target_valid11,
)
copied_source_mask11, copied_target_mask11 = experimental11.build_masks(
    source_valid11, target_valid11,
)
assert torch.equal(copied_source_mask11, source_mask11)
print("Shorter target, query 1, 1 means the key is allowed:")
print("  Keys-only:", copied_target_mask11[1, 0, 0].int().tolist())
print("  Causal:   ", causal_mask11[1, 0, 0].int().tolist())
```

OUTPUT

```
Shorter target, query 1, 1 means the key is allowed:
  Keys-only: [1, 1, 1, 0]
  Causal:    [1, 0, 0, 0]
```

```
changed_target11 = target11.clone()
changed_target11[0, 2] = 6

with torch.no_grad():
    leaky_before11 = keys_only11(source11, target11)[0, :2]
    leaky_after11 = keys_only11(source11, changed_target11)[0, :2]
    causal_before11 = causal11(source11, target11)[0, :2]
    causal_after11 = causal11(source11, changed_target11)[0, :2]

torch.testing.assert_close(
    causal_after11, causal_before11, atol=1e-12, rtol=0,
)
leaky_delta11 = (leaky_after11 - leaky_before11).abs().amax(dim=-1)
causal_delta11 = (
    causal_after11 - causal_before11
).abs().amax(dim=-1)
assert torch.all(leaky_delta11 > 1e-12)
print("Query position  Keys-only max change  Causal max change")
for i11, (leak11, safe11) in enumerate(
    zip(leaky_delta11, causal_delta11), start=1,
):
    print(f"{i11:14d}  {leak11:20.3e}  {safe11:17.3e}")
```

OUTPUT

```
Query position  Keys-only max change  Causal max change
             1             2.736e-01          0.000e+00
             2             4.676e-01          0.000e+00
```

Changing the third input changes the key and value vectors computed from that token. Without the triangle, an earlier query can assign a different weight to that key and receive a different contribution from that value. The altered self-attention result passes through the decoder and changes the vocabulary logits. With the triangle restored, that route is closed. Each early query receives exactly the inputs it was permitted to read before the edit.

The companion check tests both removing later inputs and changing a future target ID, as well as editing source padding under an explicit mask. The keys-only version fails the check at the comparison between a truncated prefix and the same prefix inside the full sequence.

```
check_prefix_and_padding(causal11, source11, target11)
print("Canonical causal model passes the companion check.")
with torch.no_grad():
    short_prefix11 = keys_only11(source11, target11[:, :2])[0]
prefix_gap11 = (short_prefix11 - leaky_before11).abs().max().item()
assert prefix_gap11 > 1e-12
print(f"Keys-only, prefix alone vs. full: {prefix_gap11:.3e}")
try:
    check_prefix_and_padding(keys_only11, source11, target11)
except AssertionError:
    print("Keys-only model is rejected by the companion check.")
else:
    raise AssertionError("The noncausal model should fail.")
```

```
OUTPUT
Canonical causal model passes the companion check.
Keys-only, prefix alone vs. full: 6.445e-01
Keys-only model is rejected by the companion check.
```

## 12. Build a deeper model (optional challenge)

A stack of two layers applies two transformations in sequence. Each layer has its own learned parameters. When a parameter has the same name in both layers, compare their Python identities and their first-element addresses using `data_ptr()`. In this model the corresponding arrays are separate tensors in separate storage allocations.

A LayerNorm scale starts at one in every coordinate and its offset starts at zero. Equal initial values describe the entries. The identity and storage comparisons establish that changing one layer's parameter need not change the other's.

```
deep12 = make_exercise_transformer(layers=2)
source12, target12, labels12 = example_batch()
stacks12 = (
    ("Encoder", deep12.encoder_layers),
    ("Decoder", deep12.decoder_layers),
)
for stack_name12, layers12 in stacks12:
    first12 = dict(layers12[0].named_parameters())
    second12 = dict(layers12[1].named_parameters())
    assert first12.keys() == second12.keys()

    for name12, parameter12 in first12.items():
        other12 = second12[name12]
        assert parameter12 is not other12, name12
        assert parameter12.data_ptr() != other12.data_ptr()
        assert (
            parameter12.untyped_storage().data_ptr()
            != other12.untyped_storage().data_ptr()
        ), name12

    for name12 in ("norm1.weight", "norm1.bias"):
        assert torch.equal(first12[name12], second12[name12])
    print(f"{stack_name12}: {len(first12)} independent pairs")
    print("  First LayerNorm scales and offsets match in value.")

check_prefix_and_padding(deep12, source12, target12)
print("Two-layer stacks pass the prefix and padding check.")
```

OUTPUT

```
Encoder: 12 independent pairs
  First LayerNorm scales and offsets match in value.
Decoder: 18 independent pairs
  First LayerNorm scales and offsets match in value.
Two-layer stacks pass the prefix and padding check.
```

The encoder first replaces its hidden array once per layer. The second encoder layer's output becomes the final memory, with shape $[2, 5, 4]$. The decoder loop updates the target array at every layer and passes that same memory tensor to every layer. Each cross-attention module uses its own learned projections to turn the shared memory into keys and values. The queries come from the decoder's current target states.

Temporary hooks observe the tensors that enter and leave these modules during the forward pass. Capture the final encoder output, then check that both decoder layers' cross-attention modules receive that very object as their key and value inputs. The hooks observe the tensors without modifying the computation.

```
final_memory12 = []
cross_inputs12 = []

def record_encoder_output12(module, inputs, output):
    final_memory12.append(output[0])

def record_cross_inputs12(module, inputs):
    cross_inputs12.append((inputs[1], inputs[2]))

hooks12 = [
    deep12.encoder_layers[-1].register_forward_hook(
        record_encoder_output12,
    )
]
for layer12 in deep12.decoder_layers:
    hooks12.append(
        layer12.cross_attention.register_forward_pre_hook(
            record_cross_inputs12,
        )
    )

try:
    logits12, attention12 = deep12(
        source12, target12, return_weights=True,
    )
finally:
    for hook12 in hooks12:
        hook12.remove()

assert len(final_memory12) == 1
assert len(cross_inputs12) == 2
memory12 = final_memory12[0]
for key12, value12 in cross_inputs12:
    assert key12 is memory12 and value12 is memory12
print("Final encoder memory shape:", tuple(memory12.shape))
print("Both decoder layers receive it as keys and values.")
print("Cross-attention shapes:",
      [tuple(a.shape) for a in attention12["cross"]])
```

OUTPUT

```
Final encoder memory shape: (2, 5, 4)
Both decoder layers receive it as keys and values.
Cross-attention shapes: [(2, 2, 4, 5), (2, 2, 4, 5)]
```

The forward pass was in evaluation mode and outside `torch.no_grad()`. This leaves autograd active and keeps dropout disabled. Flatten the batch and target axes, compute mean cross-entropy over the seven non-PAD labels, and call `backward()`. Check that each parameter receives a finite gradient. A nonzero gradient in the source embeddings gives a direct check that this target loss can flow back through cross-attention and the encoder.

```
deep12.zero_grad(set_to_none=True)
loss12 = F.cross_entropy(
    logits12.reshape(-1, 12),
    labels12.flatten(),
    ignore_index=deep12.pad_id,
)
assert torch.isfinite(loss12)
loss12.backward()

count12 = 0
for name12, parameter12 in deep12.named_parameters():
    assert parameter12.grad is not None, name12
    assert torch.isfinite(parameter12.grad).all(), name12
    count12 += 1
source_grad12 = deep12.source_embedding.weight.grad.abs().sum()
assert source_grad12 > 0

print("Logits shape:", tuple(logits12.shape))
print(f"Masked loss: {loss12.item():.6f}")
print("Parameter tensors with finite gradients:", count12)
print(f"Source embedding gradient L1 sum: {source_grad12.item():.6f}")
```

OUTPUT

```
Logits shape: (2, 4, 12)
Masked loss: 2.474837
Parameter tensors with finite gradients: 64
Source embedding gradient L1 sum: 0.065284
```